

School of Computer Science and Engineering

Fall Semester 2025-26

BCSE331P – Exploratory Data Analysis

BCSE331P – Exploratory Data Analysis

Assessment 3

Data Cleaning and Univariate Analysis

NAME : ARJUN MISHRA

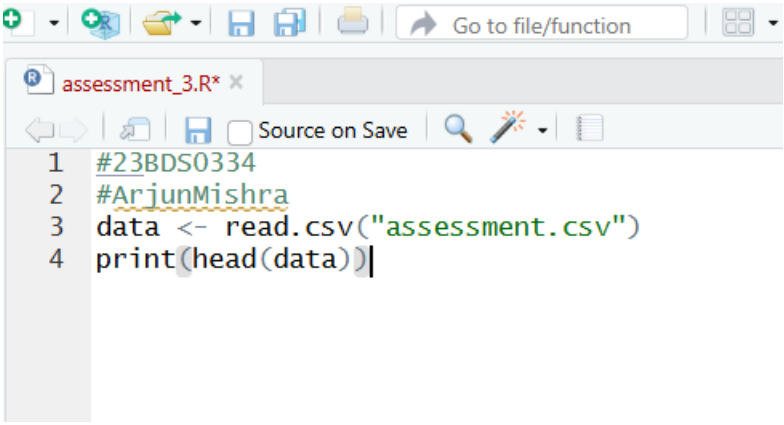
REG NO: 23BDS0334

1. **Import a dataset into R. (Dataset shared through e-mail)**

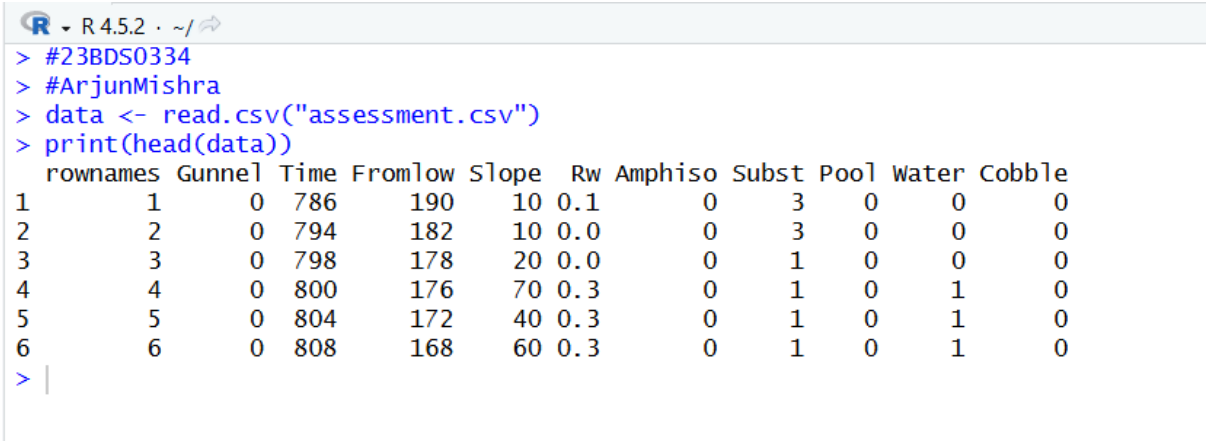
AIM:

To import the given CSV dataset into R.

CODE:



OUTPUT:



RESULT:

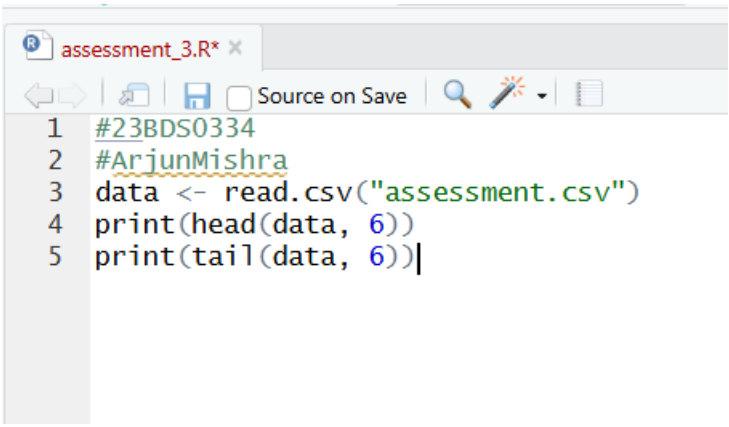
Thus, the dataset was successfully imported into R.

2. **Display the first and last six records of the dataset**

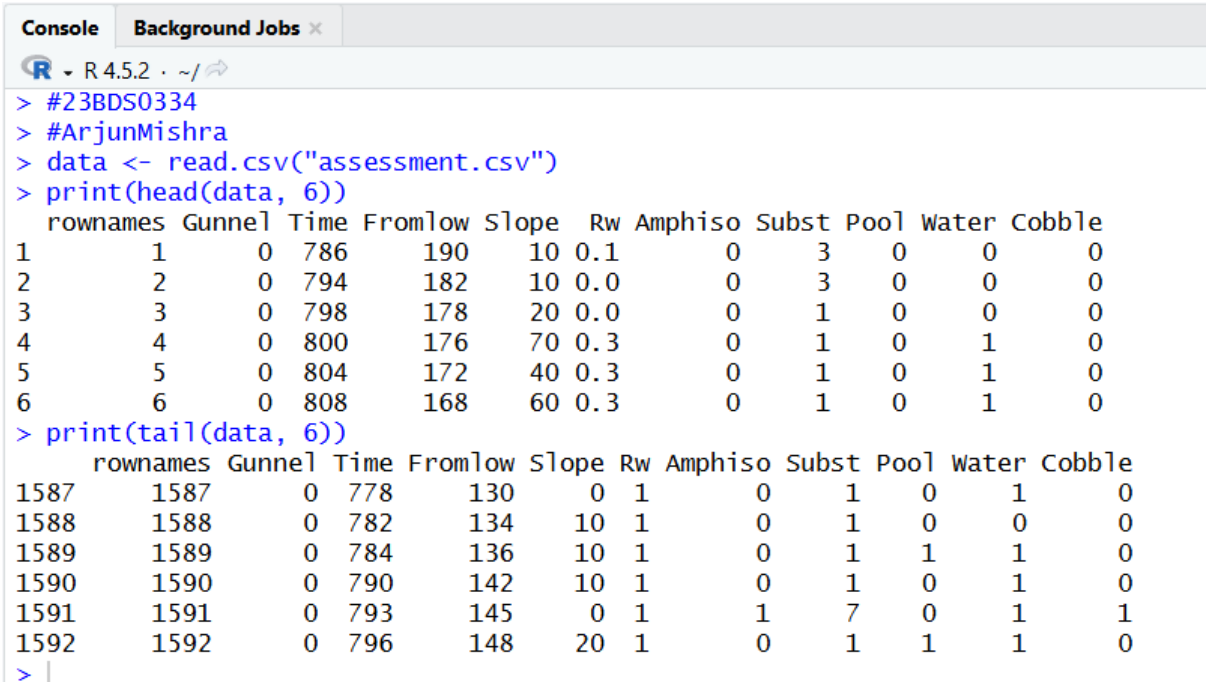
AIM:

To display the first and last six records of the dataset.

CODE:



OUTPUT:



RESULT:

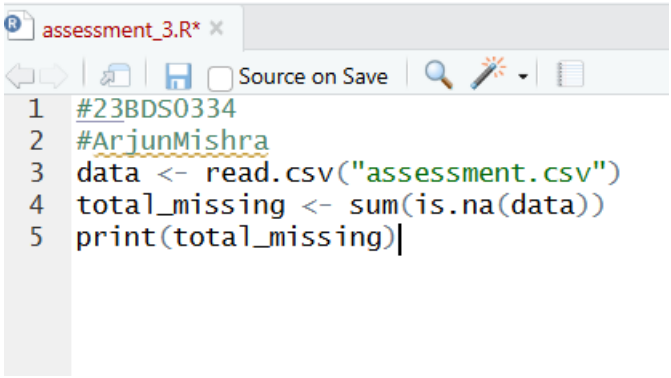
Thus, the first and last six records were successfully displayed.

3. **Identify the total number of missing values in the dataset.**

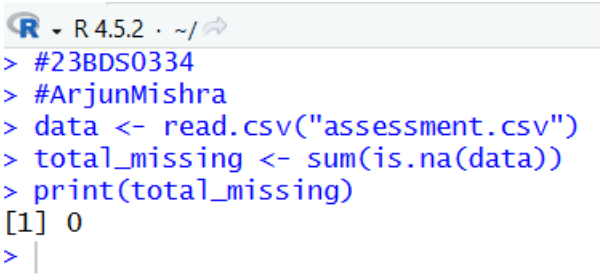
AIM:

To identify the total number of missing values in the dataset.

CODE:



OUTPUT:



RESULT:

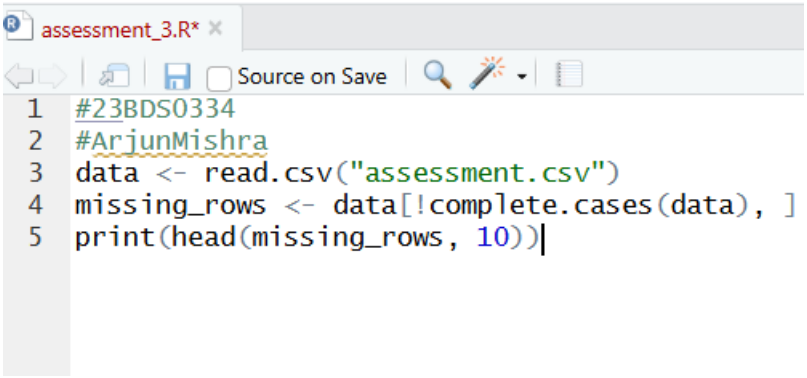
Thus, the total number of missing values was successfully identified.

4. **Display all records containing at least one missing value.**

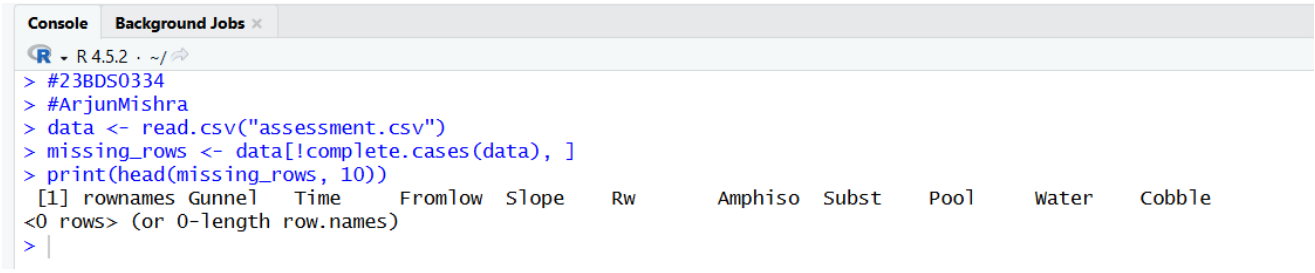
AIM:

To display records containing at least one missing value.

CODE:



OUTPUT:



RESULT:

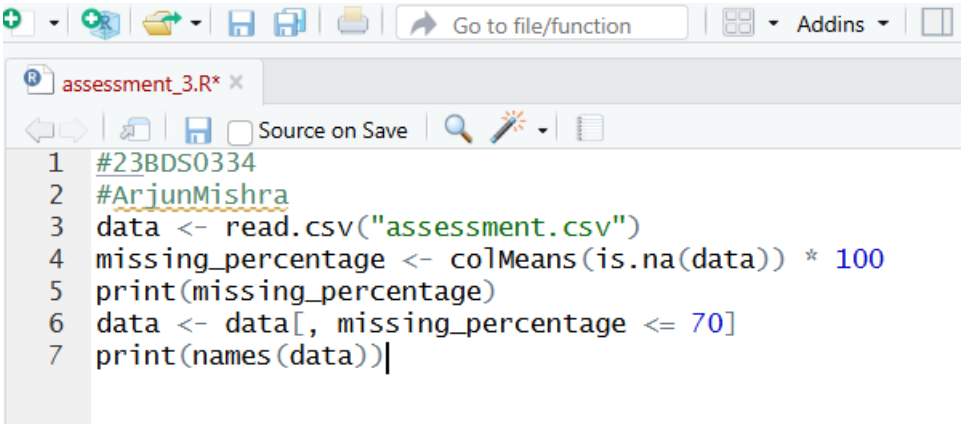
Thus, records containing missing values were successfully identified.

5. Remove columns having more than 70 percentage of missing values.

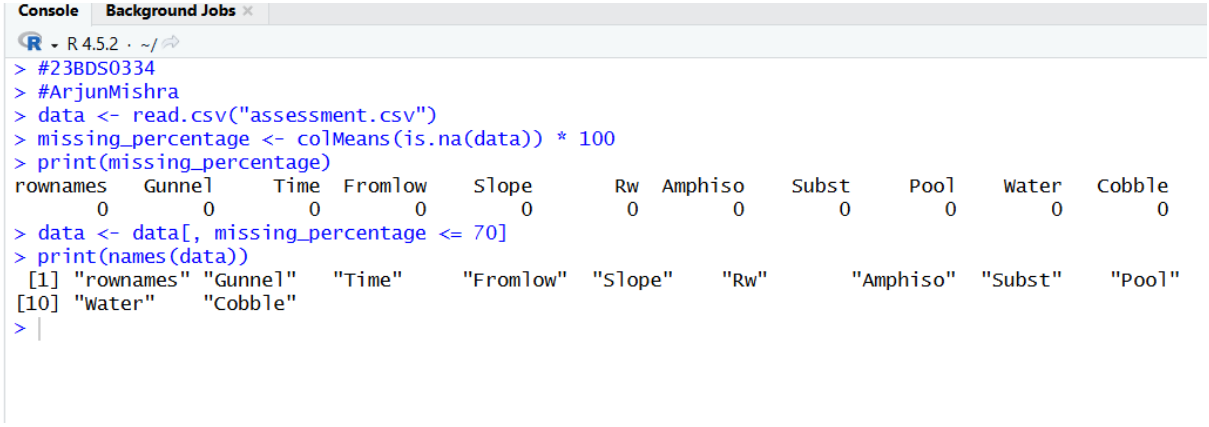
AIM:

To remove columns containing more than 70% missing values.

CODE:



OUTPUT:



RESULT:

Thus, columns with more than 70% missing values were successfully removed.

6. Replace missing numerical values with the mean.

AIM:

To replace missing numerical values with the mean.

CODE:

assessment_3.R*

Source on Save

```
#23BDS0334
#ArjunMishra
data <- read.csv("assessment.csv")
num_cols <- sapply(data, is.numeric)

data[num_cols] <- lapply(data[num_cols], function(x) {
  x[is.na(x)] <- mean(x, na.rm = TRUE)
  x
})
print(head(data, 10))
```

OUTPUT:

Console | Background Jobs

R 4.5.2 · ~/

```
> #23BDS0334
> #ArjunMishra
> data <- read.csv("assessment.csv")
> num_cols <- sapply(data, is.numeric)
>
> data[num_cols] <- lapply(data[num_cols], function(x) {
+   x[is.na(x)] <- mean(x, na.rm = TRUE)
+   x
+ })
> print(head(data, 10))
   rownames Gunnel Time Fromlow Slope  Rw Amphiso Subst Pool Water Cobble
1         1      0  786     190    10 0.1       0     3    0     0      0
2         2      0  794     182    10 0.0       0     3    0     0      0
3         3      0  798     178    20 0.0       0     1    0     0      0
4         4      0  800     176    70 0.3       0     1    0     1      0
5         5      0  804     172    40 0.3       0     1    0     1      0
6         6      0  808     168    60 0.3       0     1    0     1      0
7         7      0  814     162    40 0.3       0     1    0     1      0
8         8      0  817     159     0 0.9       1     1    1     1      0
9         9      0  821     155    50 0.5       0     1    0     1      0
10       10      0  826     150    20 0.1       0     1    0     0      0
>
```

RESULT:

Thus, missing numerical values were successfully replaced using the mean.

7. Replace missing numerical values with the median.

AIM:

To replace missing numerical values with the median.

CODE:

assessment_3.R*

Source on Save

```
#23BDS0334
#ArjunMishra
data <- read.csv("assessment.csv")
num_cols <- sapply(data, is.numeric)
data[num_cols] <- lapply(data[num_cols], function(x) {
  x[is.na(x)] <- median(x, na.rm = TRUE)
  x
})
print(head(data, 10))
```

OUTPUT:

```
> #23BDS0334
> #ArjunMishra
> data <- read.csv("assessment.csv")
> num_cols <- sapply(data, is.numeric)
> data[num_cols] <- lapply(data[num_cols], function(x) {
+   x[is.na(x)] <- median(x, na.rm = TRUE)
+   x
+ })
> print(head(data, 10))
   rownames Gunnel Time Fromlow Slope  Rw Amphiso Subst Pool Water Cobble
1         1      0  786     190    10 0.1       0     3    0     0      0
2         2      0  794     182    10 0.0       0     3    0     0      0
3         3      0  798     178    20 0.0       0     1    0     0      0
4         4      0  800     176    70 0.3       0     1    0     1      0
5         5      0  804     172    40 0.3       0     1    0     1      0
6         6      0  808     168    60 0.3       0     1    0     1      0
7         7      0  814     162    40 0.3       0     1    0     1      0
8         8      0  817     159     0 0.9       1     1    1     1      0
9         9      0  821     155    50 0.5       0     1    0     1      0
10       10      0  826     150    20 0.1       0     1    0     0      0
>
```

RESULT:

Thus, missing numerical values were successfully replaced using the median.

8. Replace missing categorical values with the mode.

AIM:

To replace missing categorical values with the mode.

CODE:

assessment_3.R*

```r
#23BDS0334
#ArjunMishra
data <- read.csv("assessment.csv")
cat_cols <- sapply(data, function(x) is.character(x) || is.factor(x))

get_mode <- function(x) {
  x2 <- x[!is.na(x)]
  names(sort(table(x2), decreasing = TRUE))[1]
}

data[cat_cols] <- lapply(data[cat_cols], function(x) {
  x[is.na(x)] <- get_mode(x)
  x
})
print(head(data, 10))
```

OUTPUT:

```
R 4.5.2 · ~/
> #23BDS0334
> #ArjunMishra
> data <- read.csv("assessment.csv")
> cat_cols <- sapply(data, function(x) is.character(x) || is.factor(x))
>
> get_mode <- function(x) {
+   x2 <- x[!is.na(x)]
+   names(sort(table(x2), decreasing = TRUE))[1]
+ }
>
> data[cat_cols] <- lapply(data[cat_cols], function(x) {
+   x[is.na(x)] <- get_mode(x)
+   x
+ })
> print(head(data, 10))
   rownames Gunnel Time Fromlow Slope  Rw Amphiso Subst Pool Water Cobble
1         1      0  786     190    10 0.1       0     3    0     0      0
2         2      0  794     182    10 0.0       0     3    0     0      0
3         3      0  798     178    20 0.0       0     1    0     0      0
4         4      0  800     176    70 0.3       0     1    0     1      0
5         5      0  804     172    40 0.3       0     1    0     1      0
6         6      0  808     168    60 0.3       0     1    0     1      0
7         7      0  814     162    40 0.3       0     1    0     1      0
8         8      0  817     159     0 0.9       1     1    1     1      0
9         9      0  821     155    50 0.5       0     1    0     1      0
10       10      0  826     150    20 0.1       0     1    0     0      0
>
```

RESULT:

Thus, missing categorical values were successfully replaced using the mode.

9. Identify duplicate rows if any.

AIM:

To identify duplicate rows in the dataset.

CODE:

assessment_3.R*

Source on Save

```
#23BDS0334
#ArjunMishra
data <- read.csv("assessment.csv")
duplicates <- data[duplicated(data), ]
print(nrow(duplicates))
print(head(duplicates, 10))
```

OUTPUT:

Console | Background Jobs

R 4.5.2 · ~/

```
> #23BDS0334
> #ArjunMishra
> data <- read.csv("assessment.csv")
> duplicates <- data[duplicated(data), ]
> print(nrow(duplicates))
[1] 0
> print(head(duplicates, 10))
 [1] rownames Gunnel   Time     Fromlow  Slope    Rw       Amphiso  Subst    Pool     Water    Cobble
<0 rows> (or 0-length row.names)
>
```

RESULT:

Thus, duplicate records were successfully identified.

10. Remove the duplicate rows if any.

AIM:

To remove duplicate rows from the dataset.

CODE:

assessment_3.R*

Source on Save

```
#23BDS0334
#ArjunMishra
data <- read.csv("assessment.csv")
data <- data[!duplicated(data), ]
print(dim(data))
print(head(data, 10))
```

OUTPUT:

R 4.5.2 · ~/

```
> #23BDS0334
> #ArjunMishra
> data <- read.csv("assessment.csv")
> data <- data[!duplicated(data), ]
> print(dim(data))
[1] 1592   11
> print(head(data, 10))
   rownames Gunnel Time Fromlow Slope  Rw Amphiso Subst Pool Water Cobble
1         1      0  786     190    10 0.1       0     3    0     0      0
2         2      0  794     182    10 0.0       0     3    0     0      0
3         3      0  798     178    20 0.0       0     1    0     0      0
4         4      0  800     176    70 0.3       0     1    0     1      0
5         5      0  804     172    40 0.3       0     1    0     1      0
6         6      0  808     168    60 0.3       0     1    0     1      0
7         7      0  814     162    40 0.3       0     1    0     1      0
8         8      0  817     159     0 0.9       1     1    1     1      0
9         9      0  821     155    50 0.5       0     1    0     1      0
10       10      0  826     150    20 0.1       0     1    0     0      0
>
```

RESULT:

Thus, duplicate records were successfully removed.

11. Identify columns stored with incorrect data types.

AIM:

To identify columns stored with incorrect data types.

CODE:

assessment_3.R*

Source on Save

```
#23BDS0334
#ArjunMishra
data <- read.csv("assessment.csv")
str(data)
print(sapply(data, class))
```

OUTPUT:

R 4.5.2 · ~/

```
> #23BDS0334
> #ArjunMishra
> data <- read.csv("assessment.csv")
> str(data)
'data.frame':   1592 obs. of  11 variables:
 $ rownames: int  1 2 3 4 5 6 7 8 9 10 ...
 $ Gunnel  : int  0 0 0 0 0 0 0 0 0 0 ...
 $ Time    : int  786 794 798 800 804 808 814 817 821 826 ...
 $ Fromlow : int  190 182 178 176 172 168 162 159 155 150 ...
 $ Slope   : int  10 10 20 70 40 60 40 0 50 20 ...
 $ Rw      : num  0.1 0 0 0.3 0.3 0.3 0.3 0.9 0.5 0.1 ...
 $ Amphiso : int  0 0 0 0 0 0 0 1 0 0 ...
 $ Subst   : int  3 3 1 1 1 1 1 1 1 1 ...
 $ Pool    : int  0 0 0 0 0 0 0 1 0 0 ...
 $ Water   : int  0 0 0 1 1 1 1 1 1 0 ...
 $ Cobble  : int  0 0 0 0 0 0 0 0 0 0 ...
> print(sapply(data, class))
 rownames    Gunnel      Time   Fromlow     Slope        Rw   Amphiso     Subst      Pool     Water
"integer" "integer" "integer" "integer" "integer" "numeric" "integer" "integer" "integer" "integer"
   Cobble
"integer"
```

RESULT:

Thus, the data types of all variables were inspected so incorrectly stored variables can be identified.

12. Convert character variables into factors if any.

AIM:

To convert character variables into factors.

CODE:

assessment_3.R*

Source on Save

```
#23BDS0334
#ArjunMishra
data <- read.csv("assessment.csv")
char_cols <- sapply(data, is.character)
data[char_cols] <- lapply(data[char_cols], as.factor)
str(data)
```

OUTPUT:

R 4.5.2 · ~/

```
> #23BDS0334
> #ArjunMishra
> data <- read.csv("assessment.csv")
> char_cols <- sapply(data, is.character)
> data[char_cols] <- lapply(data[char_cols], as.factor)
> str(data)
'data.frame':   1592 obs. of  11 variables:
 $ rownames: int  1 2 3 4 5 6 7 8 9 10 ...
 $ Gunnel  : int  0 0 0 0 0 0 0 0 0 0 ...
 $ Time    : int  786 794 798 800 804 808 814 817 821 826 ...
 $ Fromlow : int  190 182 178 176 172 168 162 159 155 150 ...
 $ Slope   : int  10 10 20 70 40 60 40 0 50 20 ...
 $ Rw      : num  0.1 0 0 0.3 0.3 0.3 0.3 0.9 0.5 0.1 ...
 $ Amphiso : int  0 0 0 0 0 0 0 1 0 0 ...
 $ Subst   : int  3 3 1 1 1 1 1 1 1 1 ...
 $ Pool    : int  0 0 0 0 0 0 0 1 0 0 ...
 $ Water   : int  0 0 0 1 1 1 1 1 1 0 ...
 $ Cobble  : int  0 0 0 0 0 0 0 0 0 0 ...
>
```

RESULT:

Thus, all available character variables were successfully converted into factors.

13. Convert text values into lowercase/ uppercase.

AIM:

To convert text values into lowercase and uppercase.

CODE:

assessment_3.R*

Source on Save

```r
#23BDS0334
#ArjunMishra
data <- read.csv("assessment.csv")
char_cols <- names(data)[sapply(data, is.character)]
if (length(char_cols) > 0) {
  x <- data[[char_cols[1]]]
  print(head(str_to_lower(x), 10))
  print(head(str_to_upper(x), 10))
} else {
  print("No character column found")
}
```

OUTPUT:

Console | Background Jobs

R 4.5.2 · ~/

```
> #23BDS0334
> #ArjunMishra
> data <- read.csv("assessment.csv")
> char_cols <- names(data)[sapply(data, is.character)]
> if (length(char_cols) > 0) {
+   x <- data[[char_cols[1]]]
+   print(head(str_to_lower(x), 10))
+   print(head(str_to_upper(x), 10))
+ } else {
+   print("No character column found")
+ }
[1] "No character column found"
>
```

RESULT:

Thus, text values were successfully converted using stringr.

14. Detect outliers using boxplots.

AIM:

To detect outliers in a numerical variable using a boxplot.

CODE:

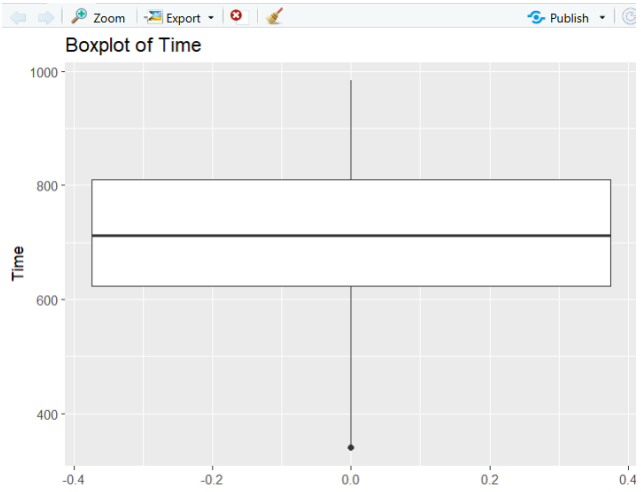
assessment_3.R*

Source on Save

```
#23BDS0334
#ArjunMishra
data <- read.csv("assessment.csv")
ggplot(data, aes(y = Time)) +
  geom_boxplot() +
  labs(title = "Boxplot of Time", y = "Time")
```

OUTPUT:

```
R 4.5.2 · ~/
> #23BDS0334
> #ArjunMishra
> data <- read.csv("assessment.csv")
> ggplot(data, aes(y = Time)) +
+   geom_boxplot() +
+   labs(title = "Boxplot of Time", y = "Time")
>
```



RESULT:

Thus, outliers in Time were successfully visualized using a boxplot.

15. Identify outliers using the interquartile range method.

AIM:

To identify outliers using the interquartile range method.

CODE:

assessment_3.R*

Source on Save

```
#23BDS0334
#ArjunMishra
data <- read.csv("assessment.csv")
Q1 <- quantile(data$Time, 0.25, na.rm = TRUE)
Q3 <- quantile(data$Time, 0.75, na.rm = TRUE)
IQR_value <- IQR(data$Time, na.rm = TRUE)
lower <- Q1 - 1.5 * IQR_value
upper <- Q3 + 1.5 * IQR_value

outliers <- data$Time[
  data$Time < lower | data$Time > upper
]
print(outliers)
```

OUTPUT:

R 4.5.2 · ~/

```
> #23BDS0334
> #ArjunMishra
> data <- read.csv("assessment.csv")
> Q1 <- quantile(data$Time, 0.25, na.rm = TRUE)
> Q3 <- quantile(data$Time, 0.75, na.rm = TRUE)
> IQR_value <- IQR(data$Time, na.rm = TRUE)
> lower <- Q1 - 1.5 * IQR_value
> upper <- Q3 + 1.5 * IQR_value
>
> outliers <- data$Time[
+   data$Time < lower | data$Time > upper
+ ]
> print(outliers)
[1] 340 341
>
```

RESULT:

Thus, outliers were successfully identified using the IQR method.

16. Remove extreme outliers from the dataset.

AIM:

To remove extreme outliers from a numerical variable.

CODE:

assessment_3.R*

Source on Save

```
#23BDS0334
#ArjunMishra
data <- read.csv("assessment.csv")
Q1 <- quantile(data$Time, 0.25, na.rm = TRUE)
Q3 <- quantile(data$Time, 0.75, na.rm = TRUE)
iqr <- IQR(data$Time, na.rm = TRUE)
lower <- Q1 - 3 * iqr
upper <- Q3 + 3 * iqr
clean_data <- data[
  is.na(data$Time) |
    (data$Time >= lower & data$Time <= upper),
]
print(dim(clean_data))
print(head(clean_data, 10))
```

OUTPUT:

```
> #ArjunMishra
> data <- read.csv("assessment.csv")
> Q1 <- quantile(data$Time, 0.25, na.rm = TRUE)
> Q3 <- quantile(data$Time, 0.75, na.rm = TRUE)
> iqr <- IQR(data$Time, na.rm = TRUE)
> lower <- Q1 - 3 * iqr
> upper <- Q3 + 3 * iqr
> clean_data <- data[
+   is.na(data$Time) |
+     (data$Time >= lower & data$Time <= upper),
+ ]
> print(dim(clean_data))
[1] 1592   11
> print(head(clean_data, 10))
   rownames Gunnel Time Fromlow Slope  Rw Amphiso Subst Pool Water Cobble
1         1      0  786     190    10 0.1       0     3    0     0      0
2         2      0  794     182    10 0.0       0     3    0     0      0
3         3      0  798     178    20 0.0       0     1    0     0      0
4         4      0  800     176    70 0.3       0     1    0     1      0
5         5      0  804     172    40 0.3       0     1    0     1      0
6         6      0  808     168    60 0.3       0     1    0     1      0
7         7      0  814     162    40 0.3       0     1    0     1      0
8         8      0  817     159     0 0.9       1     1    1     1      0
9         9      0  821     155    50 0.5       0     1    0     1      0
10       10      0  826     150    20 0.1       0     1    0     0      0
>
```

RESULT:

Thus, extreme outliers were successfully removed using the IQR rule.

17. Create a histogram for a numerical variable and interpret its distribution.

AIM:

To create a histogram for a numerical variable and study its distribution.

CODE:

assessment_3.R*

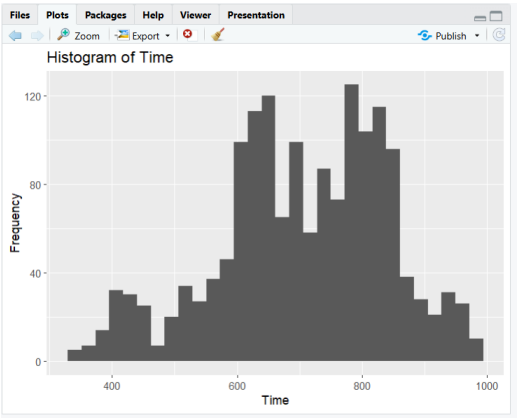
Source on Save

```
#23BDS0334
#ArjunMishra
data <- read.csv("assessment.csv")
ggplot(data, aes(x = Time)) +
  geom_histogram(bins = 30) +
  labs(
    title = "Histogram of Time",
    x = "Time",
    y = "Frequency"
  )
```

OUTPUT:

R 4.5.2 · ~/

```
> #23BDS0334
> #ArjunMishra
> data <- read.csv("assessment.csv")
> ggplot(data, aes(x = Time)) +
+   geom_histogram(bins = 30) +
+   labs(
+     title = "Histogram of Time",
+     x = "Time",
+     y = "Frequency"
+   )
```



RESULT:

Thus, the distribution of the numerical variable was successfully visualized using a histogram.

18. Add a density curve to the histogram.

AIM:

To add a density curve to a histogram.

CODE:

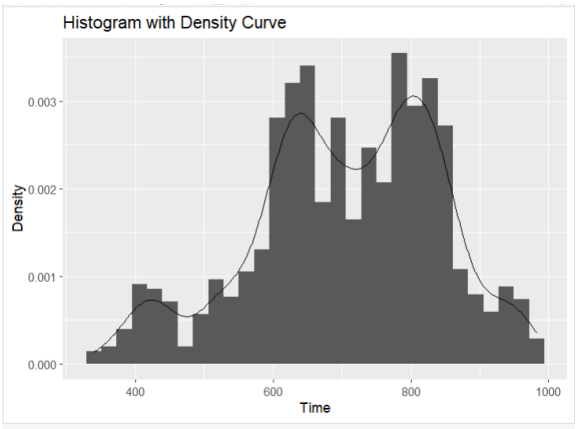
assessment_3.R*

Source on Save

```
#23BDS0334
#ArjunMishra
data <- read.csv("assessment.csv")
ggplot(data, aes(x = Time)) +
  geom_histogram(
    aes(y = after_stat(density)),
    bins = 30
  ) +
  geom_density() +
  labs(
    title = "Histogram with Density Curve",
    x = "Time",
    y = "Density"
  )
```

OUTPUT:

```
R 4.5.2 · ~/
> #23BDS0334
> #ArjunMishra
> data <- read.csv("assessment.csv")
> ggplot(data, aes(x = Time)) +
+   geom_histogram(
+     aes(y = after_stat(density)),
+     bins = 30
+   ) +
+   geom_density() +
+   labs(
+     title = "Histogram with Density Curve",
+     x = "Time",
+     y = "Density"
+   )
```



RESULT:

Thus, a density curve was successfully added to the histogram.

19. Create a violin plot.

AIM:

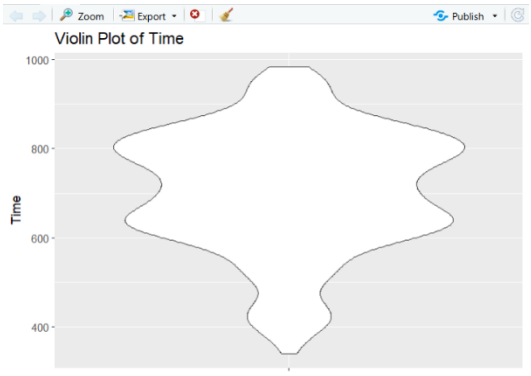
To create a violin plot for a numerical variable.

CODE:

```r
#23BDS0334
#ArjunMishra
data <- read.csv("assessment.csv")
ggplot(data, aes(x = "", y = Time)) +
  geom_violin() +
  labs(
    title = "Violin Plot of Time",
    x = "",
    y = "Time"
  )
```

OUTPUT:

```
R 4.5.2 · ~/
> #23BDS0334
> #ArjunMishra
> data <- read.csv("assessment.csv")
> ggplot(data, aes(x = "", y = Time)) +
+   geom_violin() +
+   labs(
+     title = "Violin Plot of Time",
+     x = "",
+     y = "Time"
+   )
>
```



RESULT:

Thus, a violin plot was successfully created.

20. Create a dot plot for a small numerical dataset.

AIM:

To create a dot plot for a small numerical dataset.

CODE:

assessment_3.R*

Source on Save

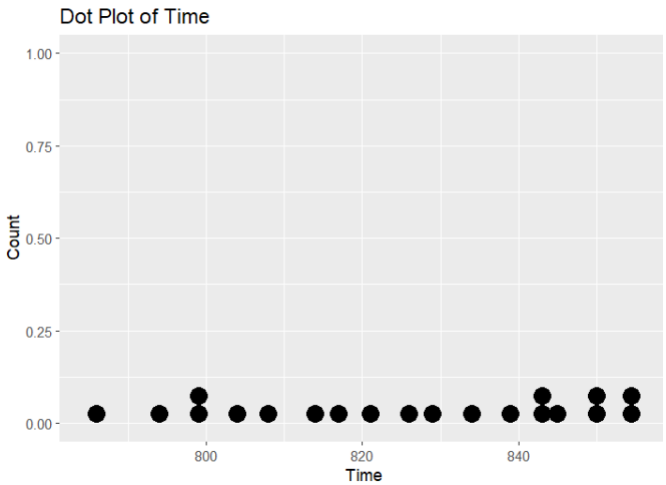
```
#23BDS0334
#ArjunMishra
data <- read.csv("assessment.csv")
small_data <- head(data, 20)
ggplot(small_data, aes(x = Time)) +
  geom_dotplot() +
  labs(
    title = "Dot Plot of Time",
    x = "Time",
    y = "Count"
  )
```

OUTPUT:

Console | Background Jobs

R 4.5.2 · ~/

```
> #23BDS0334
> #ArjunMishra
> data <- read.csv("assessment.csv")
> small_data <- head(data, 20)
> ggplot(small_data, aes(x = Time)) +
+   geom_dotplot() +
+   labs(
+     title = "Dot Plot of Time",
+     x = "Time",
+     y = "Count"
+   )
Bin width defaults to 1/30 of the range of the data. Pick better value with `binwidth`.
>
```



RESULT:

Thus, a dot plot for a small numerical dataset was successfully created.

21. Create a frequency table for a categorical variable.

AIM:

To create a frequency table for a categorical variable.

CODE:

```r
#23BDS0334
#ArjunMishra
data <- read.csv("assessment.csv")
cat_cols <- names(data)[sapply(data, function(x)
  is.character(x) || is.factor(x))]
if (length(cat_cols) > 0) {
  freq <- table(data[[cat_cols[1]]])
  print(freq)
} else {
  print("No categorical column found")
}
```

OUTPUT:

```
R 4.5.2 · ~/
> #23BDS0334
> #ArjunMishra
> data <- read.csv("assessment.csv")
> cat_cols <- names(data)[sapply(data, function(x)
+   is.character(x) || is.factor(x))]
> if (length(cat_cols) > 0) {
+   freq <- table(data[[cat_cols[1]]])
+   print(freq)
+ } else {
+   print("No categorical column found")
+ }
[1] "No categorical column found"
>
```

RESULT:

Thus, the frequency distribution of a categorical variable was successfully calculated.

22. Calculate the proportion and percentage of each category.

AIM:

To calculate the proportion and percentage of each category.

CODE:

assessment_3.R*

Source on Save

```r
#23BDS0334
#ArjunMishra
data <- read.csv("assessment.csv")
cat_cols <- names(data)[sapply(data, function(x)
  is.character(x) || is.factor(x))]
if (length(cat_cols) > 0) {
  freq <- table(data[[cat_cols[1]]])
  proportion <- prop.table(freq)
  percentage <- prop.table(freq) * 100
  print(proportion)
  print(percentage)
} else {
  print("No categorical column found")
}
```

OUTPUT:

R 4.5.2 · ~/

```
> #23BDS0334
> #ArjunMishra
> data <- read.csv("assessment.csv")
> cat_cols <- names(data)[sapply(data, function(x)
+   is.character(x) || is.factor(x))]
> if (length(cat_cols) > 0) {
+   freq <- table(data[[cat_cols[1]]])
+   proportion <- prop.table(freq)
+   percentage <- prop.table(freq) * 100
+   print(proportion)
+   print(percentage)
+ } else {
+   print("No categorical column found")
+ }
[1] "No categorical column found"
>
```

RESULT:

Thus, category-wise proportions and percentages were successfully calculated.

23. Create a bar chart showing the frequency of each category.

AIM:

To create a bar chart showing the frequency of each category.

CODE:

assessment_3.R*

Source on Save

```
#23BDS0334
#ArjunMishra
data <- read.csv("assessment.csv")
cat_cols <- names(data)[sapply(data, function(x
  is.character(x) || is.factor(x))]
if (length(cat_cols) > 0) {
  ggplot(data, aes(x = .data[[cat_cols[1]]])) +
    geom_bar() +
    labs(
      title = "Frequency Bar Chart",
      x = cat_cols[1],
      y = "Frequency"
    )
}
```

OUTPUT:

R 4.5.2 · ~/

```
> #23BDS0334
> #ArjunMishra
> data <- read.csv("assessment.csv")
> cat_cols <- names(data)[sapply(data, function(x)
+   is.character(x) || is.factor(x))]
> if (length(cat_cols) > 0) {
+   ggplot(data, aes(x = .data[[cat_cols[1]]])) +
+     geom_bar() +
+     labs(
+       title = "Frequency Bar Chart",
+       x = cat_cols[1],
+       y = "Frequency"
+     )
+ }
>
```

RESULT:

Thus, a frequency bar chart was successfully created.

24. Create a horizontal bar chart for a variable with several categories.

AIM:

To create a horizontal bar chart for a categorical variable.

CODE:

assessment_3.R* ×

Source on Save

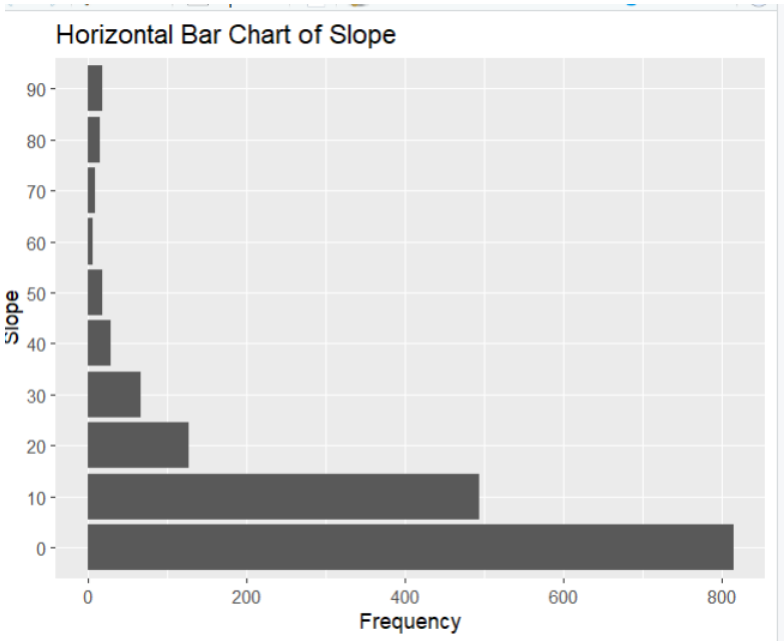
```
#23BDS0334
#ArjunMishra
library(ggplot2)

data <- read.csv("assessment.csv")

data$Slope <- as.factor(data$Slope)

ggplot(data, aes(x = Slope)) +
  geom_bar() +
  coord_flip() +
  labs(
    title = "Horizontal Bar Chart of Slope",
    x = "Slope",
    y = "Frequency"
  )
```

OUTPUT:



RESULT:

Thus, the frequency distribution of Slope was successfully represented using a horizontal bar chart.

25. Create a percentage bar chart instead of a frequency bar chart.

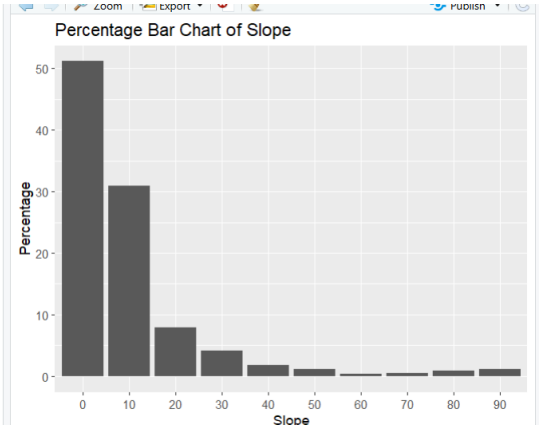
AIM:

CODE:

```
#23BDS0334
#ArjunMishra
library(dplyr)
library(ggplot2)
data <- read.csv("assessment.csv")
data$Slope <- as.factor(data$Slope)
plot_data <- data %>%
  count(Slope) %>%
  mutate(Percentage = n / sum(n) * 100)
print(head(plot_data, 10))
ggplot(plot_data, aes(x = Slope, y = Percentage)) +
  geom_col() +
  labs(
    title = "Percentage Bar Chart of Slope",
    x = "Slope",
    y = "Percentage"
  )
```

OUTPUT:

```
> #23BDS0334
> #ArjunMishra
> library(dplyr)
> library(ggplot2)
> data <- read.csv("assessment.csv")
> data$Slope <- as.factor(data$Slope)
> plot_data <- data %>%
+   count(Slope) %>%
+   mutate(Percentage = n / sum(n) * 100)
> print(head(plot_data, 10))
   Slope   n Percentage
1      0 814 51.1306533
2     10 493 30.9673367
3     20 127  7.9773869
4     30  66  4.1457286
5     40  28  1.7587940
6     50  18  1.1306533
7     60   6  0.3768844
8     70   8  0.5025126
9     80  14  0.8793970
10    90  18  1.1306533
> ggplot(plot_data, aes(x = Slope, y = Percentage)) +
+   geom_col() +
+   labs(
+     title = "Percentage Bar Chart of Slope",
+     x = "Slope",
+     y = "Percentage"
+   )
>
```



RESULT:

Thus, the percentage distribution of Slope categories was successfully represented using a percentage bar chart.

26. Create a pie chart for a categorical variable with a small number of categories.

AIM:

To create a pie chart for a categorical variable with a small number of categories.

CODE:

assessment_3.R*

Source on Save

```
#23BDS0334
#ArjunMishra
library(dplyr)
library(ggplot2)

data <- read.csv("assessment.csv")

data$Slope <- as.factor(data$Slope)

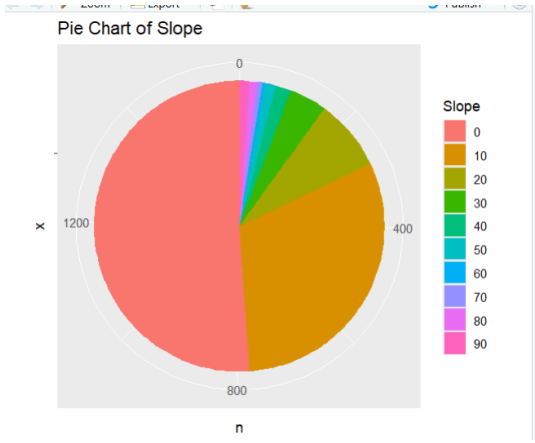
pie_data <- data %>%
  count(Slope)

print(head(pie_data, 10))

ggplot(pie_data, aes(x = "", y = n, fill = Slope)) +
  geom_col(width = 1) +
  coord_polar(theta = "y") +
  labs(
    title = "Pie Chart of Slope",
    fill = "Slope"
  )
```

OUTPUT:

```
> data <- read.csv("assessment.csv")
>
> data$Slope <- as.factor(data$Slope)
>
> pie_data <- data %>%
+   count(Slope)
>
> print(head(pie_data, 10))
   Slope   n
1      0 814
2     10 493
3     20 127
4     30  66
5     40  28
6     50  18
7     60   6
8     70   8
9     80  14
10    90  18
>
> ggplot(pie_data, aes(x = "", y = n, fill = Slope)) +
+   geom_col(width = 1) +
+   coord_polar(theta = "y") +
+   labs(
+     title = "Pie Chart of Slope",
+     fill = "Slope"
+   )
```



RESULT:

Thus, a pie chart was successfully created to represent the categorical frequency distribution.

27. Create a lollipop chart for categorical frequencies.

AIM:

To create a lollipop chart for categorical frequencies.

CODE:

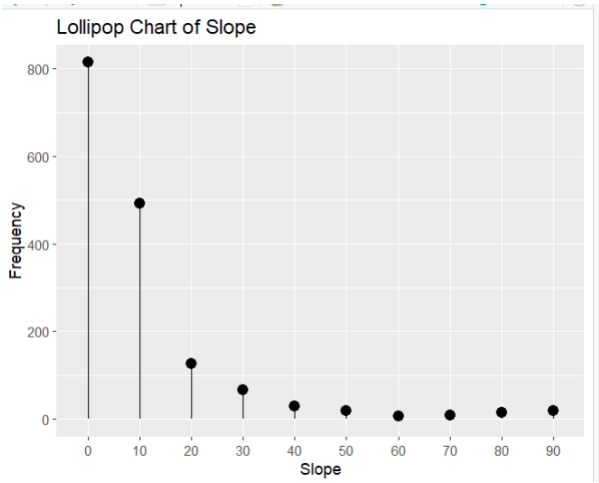
assessment_3.R*

Source on Save

```
#23BDS0334
#ArjunMishra
library(ggplot2)
data <- read.csv("assessment.csv")
data$Slope <- as.factor(data$Slope)
freq_data <- as.data.frame(table(data$Slope))
names(freq_data) <- c("Slope", "Frequency")
print(head(freq_data, 10))
ggplot(freq_data, aes(x = Slope, y = Frequency)) +
  geom_segment(
    aes(xend = Slope, y = 0, yend = Frequency)
  ) +
  geom_point(size = 3) +
  labs(
    title = "Lollipop Chart of Slope",
    x = "Slope",
    y = "Frequency"
  )
```

OUTPUT:

```
> #23BDS0334
> #ArjunMishra
> library(ggplot2)
> data <- read.csv("assessment.csv")
> data$Slope <- as.factor(data$Slope)
> freq_data <- as.data.frame(table(data$Slope))
> names(freq_data) <- c("Slope", "Frequency")
> print(head(freq_data, 10))
   Slope Frequency
1      0       814
2     10       493
3     20       127
4     30        66
5     40        28
6     50        18
7     60         6
8     70         8
9     80        14
10    90        18
> ggplot(freq_data, aes(x = Slope, y = Frequency)) +
+   geom_segment(
+     aes(xend = Slope, y = 0, yend = Frequency)
+   ) +
+   geom_point(size = 3) +
+   labs(
+     title = "Lollipop Chart of Slope",
+     x = "Slope",
+     y = "Frequency"
+   )
```



RESULT:

Thus, the frequency distribution of the Slope categories was successfully represented using a lollipop chart.

28. Create a dot chart

AIM:

To create a dot chart.

CODE:

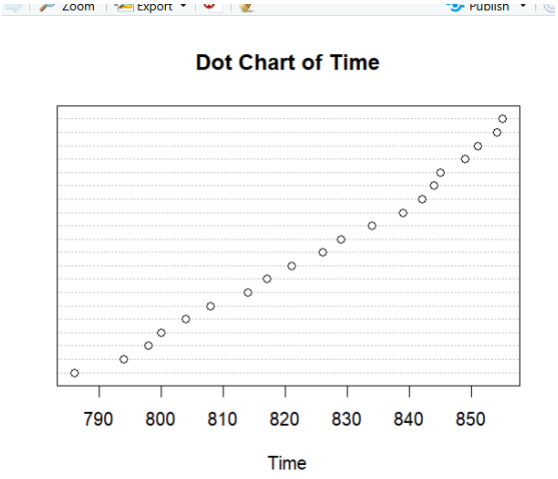
assessment_3.R*

Source on Save

```r
#23BDS0334
#ArjunMishra
library(ggplot2)
data <- read.csv("assessment.csv")
values <- head(na.omit(data$Time), 20)
dotchart(
  values,
  main = "Dot Chart of Time",
  xlab = "Time"
)
```

OUTPUT:

```
R 4.5.2 · ~/
> #23BDS0334
> #ArjunMishra
> library(ggplot2)
> data <- read.csv("assessment.csv")
> values <- head(na.omit(data$Time), 20)
> dotchart(
+   values,
+   main = "Dot Chart of Time",
+   xlab = "Time"
+ )
>
```



RESULT:

Thus, a dot chart was successfully created.

29. Create a density plot using ggplot2.

AIM:

To create a density plot for a numerical variable using ggplot2.

CODE:

assessment_3.R*

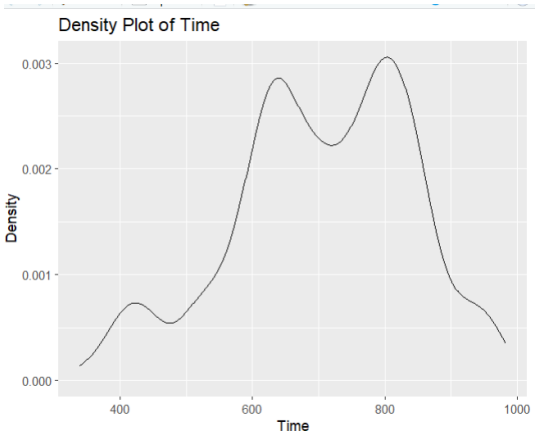
Source on Save

```
#23BDS0334
#ArjunMishra
library(ggplot2)
data <- read.csv("assessment.csv")
ggplot(data, aes(x = Time)) +
  geom_density(na.rm = TRUE) +
  labs(
    title = "Density Plot of Time",
    x = "Time",
    y = "Density"
  )
```

OUTPUT:

R 4.5.2 · ~/

```
#23BDS0334
#ArjunMishra
library(ggplot2)
data <- read.csv("assessment.csv")
ggplot(data, aes(x = Time)) +
  geom_density(na.rm = TRUE) +
  labs(
    title = "Density Plot of Time",
    x = "Time",
    y = "Density"
  )
```



RESULT:

Thus, the density distribution of the numerical variable was successfully visualized using ggplot2.

30. Use facet_wrap() to display the distribution of one variable across different groups while interpreting each distribution separately.

AIM:

To use facet_wrap() to display the distribution of one numerical variable across different groups.

CODE:

assessment_3.R*

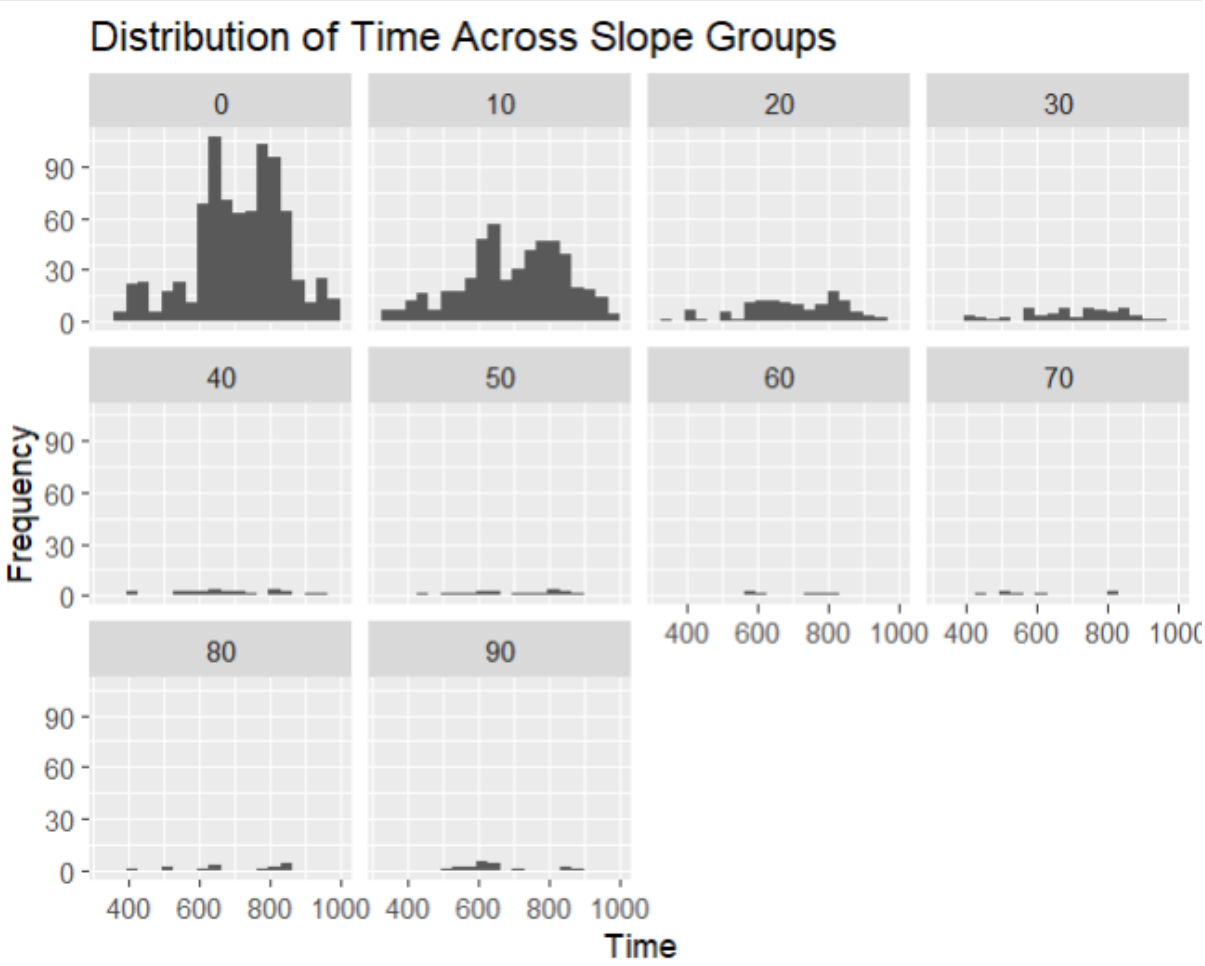
Source on Save

```
#23BDS0334
#ArjunMishra
library(ggplot2)
data <- read.csv("assessment.csv")
data$Slope <- as.factor(data$Slope)
ggplot(data, aes(x = Time)) +
  geom_histogram(bins = 20) +
  facet_wrap(~Slope) +
  labs(
    title = "Distribution of Time Across Slope Groups",
    x = "Time",
    y = "Frequency"
  )
```

OUTPUT:

R 4.5.2 · ~/

```
> #23BDS0334
> #ArjunMishra
> library(ggplot2)
> data <- read.csv("assessment.csv")
> data$Slope <- as.factor(data$Slope)
> ggplot(data, aes(x = Time)) +
+   geom_histogram(bins = 20) +
+   facet_wrap(~Slope) +
+   labs(
+     title = "Distribution of Time Across Slope Groups",
+     x = "Time",
+     y = "Frequency"
+   )
>
```



RESULT:

Thus, facet_wrap() successfully displayed the distribution of a numerical variable across different groups, allowing each group's distribution to be interpreted separately.